

ARRAYS e FOREACH EM C#

O que vais aprender:

1. Porquê é que os arrays existem (e que dor de cabeça resolvem)
2. O que acontece na memória do computador
3. Como criar um array
4. Como ler e alterar valores
5. Percorrer arrays com for — aqui fica fixe a sério
6. O foreach — uma forma mais simples de percorrer
7. Exercícios finais para consolidar tudo

Porquê é que os arrays existem?

Imagina que és o diretor de turma e queres guardar as notas de 5 alunos num programa. Provavelmente farias isto:

```
// A forma ingénua — uma variável por aluno
int notaAna    = 15;
int notaBruno  = 12;
int notaCarlos = 18;
int notaDiana  = 9;
int notaEva    = 14;
```

Parece razoável para 5 alunos. Mas e se a turma tiver 30? Ou 100? Já imaginas o trabalho...

E para calcular a média tinhas de fazer assim:

```
// Calcular a média sem arrays — horrível
double media = (notaAna + notaBruno + notaCarlos + notaDiana + notaEva) / 5.0;
// Com 30 alunos isto seria uma linha enorme e ridícula...
```

 **A solução chama-se array.** Um array é como uma caixa com gavetas numeradas. Em vez de 30 variáveis diferentes, tens uma só variável com 30 espaços dentro.

Com um array, o mesmo programa fica assim:

```
// Com array — muito melhor
int[] notas = { 15, 12, 18, 9, 14 };

// Para mostrar todas as notas — já vês como isto funciona mais à frente!
// (spoiler: é um ciclo for de 3 linhas)
```

 **Resumindo:** Array = uma variável que guarda vários valores do mesmo tipo, organizados e numerados.

Exercício — Secção 1

Exercício 1 — Antes de continuar, pensa

1. Se quisesse guardar as temperaturas de cada hora do dia (24 horas), quantas variáveis precisavas sem arrays? E com um array?
2. Dá dois exemplos do teu dia-a-dia onde uma lista de valores do mesmo tipo faz sentido.

O que acontece na memória do computador?

Não precisas de ser especialista em hardware para perceber isto — mas ajuda muito saber o básico.

A memória como uma fila de caixas

Imagina a memória RAM como uma fila enorme de caixas numeradas, uma a seguir à outra. Cada caixa guarda um único valor.

Quando crias um array de 5 elementos, o computador reserva 5 caixas SEGUIDAS na memória:

15	12	18	9	14
[0]	[1]	[2]	[3]	[4]

Cada caixa é uma posição do array. O número em baixo é o índice.

O elemento destacado (18) está na posição de índice 2 — a terceira caixa, contando a partir do zero.

Porquê começar no índice 0 e não no 1?

Esta é a dúvida de toda a gente ao início. A resposta é simples:

- O índice representa quantas posições estás a contar a partir do início.
- O primeiro elemento está a ZERO posições do início → índice 0.
- O segundo está a UMA posição do início → índice 1.
- E assim por diante.

O erro mais comum de sempre

🚫 **IndexOutOfRangeException** Se o array tem 5 elementos, os índices válidos são 0, 1, 2, 3 e 4. Tentar aceder ao índice 5 (que não existe) dá um erro e o programa fecha. É o erro mais comum em programação com arrays — toda a gente o comete pelo menos uma vez.

```
// Cuidado com os limites!
int[] notas = { 15, 12, 18, 9, 14 }; // 5 elementos: índices 0 a 4

Console.WriteLine(notas[0]); // ✅ OK — primeiro elemento
Console.WriteLine(notas[4]); // ✅ OK — último elemento
Console.WriteLine(notas[5]); // ❌ ERRO — índice 5 não existe!
```

Exercício — Secção 2

Exercício 2 — Raciocínio sobre índices

3. Um array tem 8 elementos. Qual é o índice do primeiro? E do último?
4. Olha para este array: `string[] dias = { "Seg", "Ter", "Qua", "Qui", "Sex" };`
5. — Qual é o valor de `dias[2]`?
6. — Qual índice tem o valor "Sex"?
7. — O que acontece se tentares aceder a `dias[5]`?

Como criar um array

Há basicamente duas formas de criar um array. Vamos ver as duas com calma.

Forma 1 — Já sabes os valores

Quando já tens os valores que queres guardar, crias o array assim:

```
// Declarar com valores imediatos
// tipo[] nome = { valor1, valor2, valor3 };

int[]    notas  = { 15, 12, 18, 9, 14 };
string[] frutas = { "Maçã", "Banana", "Laranja" };
double[] precos = { 1.99, 3.50, 0.89 };
```

 **O [] faz toda a diferença.** `int` é um número. `int[]` é uma coleção de números. É o par de parênteses retos que diz ao C# que isto é um array.

Forma 2 — Ainda não sabes os valores

Quando sabes quantos elementos vais precisar mas ainda não tens os valores (por exemplo, porque vais pedir ao utilizador):

```
// Declarar com tamanho mas sem valores
// tipo[] nome = new tipo[tamanho];

int[]    notas = new int[5];    // 5 inteiros, todos começam a 0
string[] nomes = new string[3]; // 3 strings, todas começam como null
double[] temps = new double[7]; // 7 decimais, todos começam a 0.0
```

 **Valor inicial automático.** Quando crias um array vazio, o C# preenche tudo com um valor por omissão: inteiros ficam a 0, decimais a 0.0, strings a null e booleanos a false. Não é lixo — é tudo controlado! Caso fosse em C (uma linguagem de baixo nível), já não era bem assim...

A propriedade .Length — quantos elementos tem?

Podes sempre perguntar ao array quantos elementos tem usando `.Length`:

```
// Usar .Length
int[] notas = { 15, 12, 18, 9, 14 };

Console.WriteLine(notas.Length);           // 5
Console.WriteLine(notas[0]);               // 15 (primeiro)
Console.WriteLine(notas[notas.Length - 1]); // 14 (último - sempre!)
```

 **Para o último elemento:** Usa sempre `array[array.Length - 1]`. Assim funciona para qualquer tamanho de array, sem teres de saber o número de posições de memória.

Exercícios — Secção 3

Exercício 3 — Criar o teu primeiro array

8. Cria um array chamado `diasSemana` com os 7 dias da semana.
9. Usa `Console.WriteLine` para mostrar o primeiro e o último dia.
10. Mostra também o número total de elementos com `.Length`.

Exercício 4 — Array de temperaturas

11. Cria um array chamado `temps` com 5 posições (não sabes ainda os valores).
12. Atribui manualmente as temperaturas: 22, 25, 19, 23, 27.
13. Mostra as temperaturas nos índices 0, 2 e 4.
14. Mostra o número de elementos com `.Length`.

Exercício 5 — Encontrar o erro

15. O código abaixo tem erros. Identifica-os e corrige-os:
16. `string[] nomes = new string[3];`
17. `nomes[1] = "Ana";`
18. `nomes[2] = "Bruno";`
19. `nomes[3] = "Carlos";`
1. `Console.WriteLine(nomes.length);`

Ler e alterar elementos do array

Ler um elemento

Para ler um valor do array, usas o nome do array seguido do índice entre parênteses retos:

```
// Ler elementos pelo índice
string[] frutas = { "Maçã", "Banana", "Laranja", "Uva" };

Console.WriteLine(frutas[0]);    // Maçã
Console.WriteLine(frutas[2]);    // Laranja

// Também podes guardar numa variável
string escolha = frutas[1];
Console.WriteLine($"Escolheste: {escolha}");    // Escolheste: Banana
```

Alterar um elemento

Alterar um valor é igual a atribuir uma variável normal — só que com o índice:

```
// Alterar elementos
int[] pontos = { 100, 85, 92 };

Console.WriteLine(pontos[1]);    // 85 (antes)

pontos[1] = 95;    // atualiza o segundo elemento

Console.WriteLine(pontos[1]);    // 95 (depois)
```

Exemplo do mundo real — lista de compras

```
// Lista de compras simples
string[] lista = { "Leite", "Pão", "Ovos", "Manteiga", "Café" };

// Mostrar o primeiro e o último produto
Console.WriteLine($"Primeiro: {lista[0]}");
Console.WriteLine($"Último: {lista[lista.Length - 1]}");

// Afinal não quero manteiga — vou substituir por Queijo
lista[3] = "Queijo";

Console.WriteLine($"Produto 4 atualizado: {lista[3]}");    // Queijo
```

Exercícios — Secção 4

Exercício 6 — Leitura e escrita básica

2. Cria um array com os nomes de 4 colegas.
3. Mostra o primeiro e o último nome.
4. Altera o segundo nome para outro à tua escolha e mostra-o novamente.

Exercício 7 — Preços e atualização

5. Cria um array com 4 preços de produtos (double).
6. Mostra o preço mais caro e o mais barato — identifica-os manualmente pelos índices.
7. Aplica um desconto de 10% ao produto mais caro (multiplica por 0.90).
8. Mostra o preço antes e depois do desconto.

Arrays com ciclo for — aqui fica interessante

Até agora acedemos aos elementos do array um a um, à mão, à mão, pelo índice. Mas a verdadeira força dos arrays aparece quando os combinamos com o ciclo for.

O casamento perfeito. O índice do for (a variável *i*) serve exatamente para percorrer os índices do array. É como se tivessem sido criados um para o outro.

Percorrer todos os elementos

```
// Mostrar todos os elementos com for
string[] frutas = { "Maçã", "Banana", "Laranja", "Uva", "Manga" };

for (int i = 0; i < frutas.Length; i++)
{
    Console.WriteLine($"Fruta {i}: {frutas[i]}");
}

// Resultado:
// Fruta 0: Maçã
// Fruta 1: Banana
// Fruta 2: Laranja
// Fruta 3: Uva
// Fruta 4: Manga
```

⚠ Atenção à condição do for! Usa *i < frutas.Length* e NÃO *i <= frutas.Length*. Se o array tem 5 elementos, *.Length* é 5 e o último índice válido é 4. Com *<=* tentarias aceder ao índice 5 — erro!

Preencher o array com o utilizador

Aqui é onde as coisas ficam práticas. Em vez de pôr os valores no código, pedem-se ao utilizador:

```
// Pedir valores ao utilizador e guardar no array
int[] notas = new int[4];

// Primeiro for — para preencher
for (int i = 0; i < notas.Length; i++)
{
    Console.Write($"Nota do aluno {i + 1}: ");
    notas[i] = int.Parse(Console.ReadLine());
}

// Segundo for — para mostrar
Console.WriteLine("\n--- Notas introduzidas ---");
for (int i = 0; i < notas.Length; i++)
{
    Console.WriteLine($"Aluno {i + 1}: {notas[i]} valores");
}
```

Repara no *i + 1*. Dentro do for usamos *i + 1* para mostrar 'Aluno 1', 'Aluno 2', etc. — mas o array usa sempre o índice *i* (que começa em 0). É só para a apresentação ficar mais natural.

Calcular a soma e a média

```
// Soma e média de um array
int[] notas = { 15, 12, 18, 9, 14 };
int soma = 0;

for (int i = 0; i < notas.Length; i++)
{
    soma = soma + notas[i];    // ou: soma += notas[i];
}

double media = (double)soma / notas.Length;
Console.WriteLine($"Soma: {soma}");
Console.WriteLine($"Média: {media:F1}");    // F1 = 1 casa decimal
```

Encontrar o maior e o menor valor

Para encontrar o maior valor, comesças por assumir que o primeiro elemento é o maior. Depois percorres o resto e atualiza sempre que encontras um maior:

```
// Encontrar o maior valor
int[] temperaturas = { 22, 28, 19, 31, 25, 17, 29 };

int maior = temperaturas[0];    // assume que o primeiro é o maior

for (int i = 1; i < temperaturas.Length; i++)    // começa em 1 – o 0 já foi
{
    if (temperaturas[i] > maior)
    {
        maior = temperaturas[i];
    }
}

Console.WriteLine($"Temperatura mais alta: {maior}°C");    // 31
```

Usar if / else if / else dentro do for

Podes tomar decisões para cada elemento do array. Por exemplo, classificar notas:

```
// Classificar cada nota com if / else if / else
int[] notas = { 15, 8, 19, 10, 13, 6 };

for (int i = 0; i < notas.Length; i++)
{
    string classificacao;

    if (notas[i] >= 17)
    {
        classificacao = "Muito Bom";
    }
    else if (notas[i] >= 14)
    {
        classificacao = "Bom";
    }
    else if (notas[i] >= 10)
    {
        classificacao = "Suficiente";
    }
    else
    {
        classificacao = "Reprovado";
    }

    Console.WriteLine($"Nota {notas[i]}: {classificacao}");
}
```

Arrays paralelos — juntar dois arrays

Por vezes precisas de relacionar dois arrays pelo mesmo índice. Por exemplo, uma lista de nomes e uma lista de notas:

```
// Dois arrays — mesmo índice, informação relacionada
string[] alunos = { "Ana", "Bruno", "Carlos", "Diana" };
int[] notas = { 15, 9, 18, 12 };

// índice 0 = Ana com nota 15
// índice 1 = Bruno com nota 9, etc.

for (int i = 0; i < alunos.Length; i++)
{
    if (notas[i] >= 10)
    {
        Console.WriteLine($"{alunos[i]}: {notas[i]} — Aprovado ☒");
    }
    else
    {
        Console.WriteLine($"{alunos[i]}: {notas[i]} — Reprovado ☐");
    }
}
```

Exercícios — Secção 5

Exercício 8 — Semana de temperaturas

9. Cria um array com as temperaturas dos 7 dias da semana (valores à tua escolha).
10. Usa um for para mostrar todas as temperaturas, numeradas: 'Dia 1: 22°C', etc.
11. Calcula e mostra a média semanal.

Exercício 9 — Notas da turma

12. Pede ao utilizador 5 notas (inteiros) e guarda-as num array.
13. Usa um segundo for para mostrar todas as notas.
14. Mostra a nota mais alta e a nota mais baixa.

Exercício 10 — Classificar temperaturas

15. Array com temperaturas dos 7 dias da semana (pede ao utilizador).
16. Usa for com if/else if/else para classificar cada dia:
17. — Abaixo de 10°C: 'Frio ❄️'
18. — 10 a 19°C: 'Ameno ☀️'
19. — 20 a 29°C: 'Quente 🌞'
20. — 30°C ou mais: 'Muito quente 🔥'
21. No final, mostra quantos dias foram 'Frio'.

Exercício 11 — Lista de compras com preços

22. Dois arrays paralelos: 5 produtos e os seus preços.
23. Usa for para mostrar a lista completa.
24. Calcula o total da compra.
25. Mostra o produto mais barato e o mais caro.
26. Aplica 5% de desconto a produtos com preço superior a 10€ e mostra o preço final.

Exercício 12 — Turma — análise completa

27. Dois arrays paralelos: 6 nomes de alunos e as suas notas (pede ao utilizador).
28. Mostra a tabela completa com nome, nota e classificação (Reprovado / Suficiente / Bom / Muito Bom).
29. Mostra a média da turma.
30. Mostra o nome do melhor aluno.
31. Conta quantos alunos reprovaram (nota < 10).

O ciclo foreach — uma forma mais simples

Já sabes percorrer arrays com o for. O foreach faz a mesma coisa, mas com menos código — é como um for simplificado para quando só precisas de ler os elementos um a um.

A ideia básica

Com o for, tu controlas o índice (i) e usas-o para aceder ao array. Com o foreach, o C# faz esse trabalho por ti — vai buscar cada elemento automaticamente:

```
// for vs foreach - o mesmo resultado
string[] frutas = { "Maçã", "Banana", "Laranja" };

// COM FOR
for (int i = 0; i < frutas.Length; i++)
{
    Console.WriteLine(frutas[i]);
}

// COM FOREACH - exatamente o mesmo resultado, menos código
foreach (string fruta in frutas)
{
    Console.WriteLine(fruta);
}
```

 **Como ler o foreach:** foreach (string fruta in frutas) significa: 'Para cada string que se chama fruta, dentro do array frutas, faz isto'. A variável fruta recebe automaticamente o valor de cada elemento.

Sintaxe do foreach

```
// Estrutura
foreach (tipo nomeVariavel in nomeArray)
{
    // usa nomeVariavel para aceder ao elemento atual
}
```

Exemplos práticos

```
// Calcular a soma com foreach
int[] notas = { 15, 12, 18, 9, 14 };
int soma = 0;

foreach (int nota in notas)
{
    soma += nota;
}

Console.WriteLine($"Soma: {soma}");
Console.WriteLine($"Média: {(double)soma / notas.Length:F1}");
```

```
// Filtrar com foreach + if
int[] temperaturas = { 22, 8, 31, 15, 28, 5, 19 };

Console.WriteLine("Dias com calor (acima de 25°C):");
foreach (int temp in temperaturas)
{
    if (temp > 25)
    {
        Console.WriteLine($"{temp}°C");
    }
}
```

Quando usar for e quando usar foreach?

Situação	Usa for	Usa foreach
Precisas de saber o índice (posição) do elemento	✅ Sim	❌ Não tens o índice
Queres modificar os valores do array	✅ Sim	❌ Não funciona
Queres percorrer ao contrário	✅ Sim	❌ Não é direto
Só precisas de ler os valores, um a um	✅ Funciona	✅ Mais simples
Queres código mais curto e legível	⇒ Mais verboso	✅ Mais limpo

✅ **Regra prática:** Se precisas do índice ou de alterar valores → usa for. Se só queres ler e percorrer → foreach é mais limpo.

⚠️ **Atenção:** O foreach não te deixa alterar os valores do array dentro do ciclo. Se tentares fazer `fruta = "Kiwi"` dentro de um foreach, o C# dá erro.

Exercícios — Secção 6

Exercício 13 — Primeiro foreach

32. Cria um array com os nomes de 5 países.
33. Usa foreach para mostrar todos os países.
34. Usa outro foreach para contar quantos países têm mais de 6 letras (.Length num string dá o número de letras).

Exercício 14 — Temperaturas com foreach

35. Array com as temperaturas dos 7 dias da semana.
36. Usa foreach para calcular a soma e depois a média.
37. Usa outro foreach para mostrar apenas os dias com temperatura abaixo de 20°C.

Exercício 15 — Notas — for ou foreach?

38. Array com 6 notas de alunos.
39. Usa foreach para calcular a média.
40. Usa for (não foreach!) para encontrar a nota mais alta e mostrar em que posição está.
41. Explica num comentário no código porque é que precisas do for para a segunda parte.

7 Exercícios Finais

Estes exercícios misturam tudo: arrays, for, foreach, e if/else. **Estão organizados do mais simples para o mais difícil.**

Nível Fácil

Exercício 16 — Playlist de músicas ☆

42. Cria um array com 5 nomes de músicas.
43. Usa foreach para listar todas as músicas numeradas: '1. Nome da música'.
44. Dica: usa uma variável int contador = 1 fora do foreach e incrementa dentro.

Exercício 17 — Poupança semanal ☆

45. Cria um array com os valores que poupaste em cada dia da semana (double).
46. Usa foreach para calcular o total poupado.
47. Usa for para mostrar o dia em que poupaste mais.

Exercício 18 — Turma de natação ☆

48. Array com os tempos (em segundos) de 5 nadadores nos 100 metros.
49. Mostra todos os tempos com for.
50. Mostra o tempo mais rápido (menor valor) e o mais lento.
51. Calcula a média dos tempos.

Nível Médio

Exercício 19 — Painel meteorológico ☆☆

52. Dois arrays paralelos: nomes dos 7 dias da semana e temperaturas.
53. Usa for para mostrar: 'Segunda: 22°C — Ameno 🌤️' (com classificação).
54. Usa foreach para calcular a média.
55. Mostra o dia mais quente e o dia mais frio (com o nome do dia).

Exercício 20 — Mini caixa registadora ☆☆

56. Pede ao utilizador 5 preços de produtos e guarda num array.
57. Mostra todos os produtos numerados com o preço.
58. Calcula o total.
59. Aplica um desconto de 10% ao total se este for superior a 50€.
60. Mostra o total final com e sem desconto.

Exercício 21 — Votação ☆☆

61. Dois arrays paralelos: 4 candidatos e os seus votos.

62. Mostra a tabela completa: nome e número de votos.
63. Calcula o total de votos.
64. Mostra para cada candidato a percentagem de votos que obteve.
65. Anuncia o vencedor (quem tem mais votos).

Nível Desafio

Exercício 22 — Inventário de loja ☆☆☆

66. Três arrays paralelos: 5 produtos, quantidades em stock e preços unitários.
67. Mostra a tabela completa: produto, stock, preço e valor total em stock (quantidade × preço).
68. Mostra o valor total de todo o inventário.
69. Identifica os produtos com stock abaixo de 5 unidades e mostra um alerta.
70. Aplica desconto de 20% aos produtos com mais de 20 unidades em stock.

Exercício 23 — Analisador de turma completo ☆☆☆

71. Pede ao utilizador os nomes e notas de 6 alunos (dois arrays paralelos).
72. Mostra a tabela: nome, nota, classificação.
73. Mostra a média da turma.
74. Mostra o nome do melhor e do pior aluno.
75. Mostra quantos alunos estão acima e abaixo da média.
76. Mostra uma lista separada de aprovados e reprovados.

Exercício 24 — DESAFIO FINAL — estatísticas de vendas ☆☆☆

77. Uma loja tem 7 dias de vendas. Pede ao utilizador os valores diários.
78. Mostra todos os dias com o valor.
79. Calcula: total semanal, média diária, melhor dia, pior dia.
80. Classifica cada dia: 'Fraco' (abaixo de 50% da média), 'Normal', 'Excelente' (acima de 150% da média).
81. Mostra quantos dias foram de cada categoria.
82. Bónus: mostra se a segunda metade da semana foi melhor ou pior que a primeira.

Resumo Rápido

O quê	Como se escreve	Para quê
Criar com valores	<code>int[] a = { 1, 2, 3 };</code>	Quando já sabes os valores
Criar vazio	<code>int[] a = new int[5];</code>	Quando só sabes o tamanho
Ler um elemento	<code>a[2]</code>	Aceder ao elemento no índice 2
Alterar um elemento	<code>a[2] = 99;</code>	Mudar o valor no índice 2
Saber o tamanho	<code>a.Length</code>	Quantos elementos tem o array
Último elemento	<code>a[a.Length - 1]</code>	Sempre funciona, qualquer tamanho
Percorrer com for	<code>for (int i=0; i<a.Length; i++)</code>	Quando precisas do índice ou alterar
Percorrer com foreach	<code>foreach (int x in a)</code>	Quando só precisas de ler

⚠ **Nunca esqueças:** O índice começa em 0. O último índice válido é `array.Length - 1`. Sair desses limites dá `IndexOutOfRangeException`.